Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №16
Разработка веб-приложения на Python с использованием Django

1. Цель работы

Получить практические навыки разработки веб-приложений на Python с использованием фреймворка Django: познакомиться с архитектурой Django, создать проект и приложение, описать модели данных средствами Django ORM, выполнить миграции, реализовать связи между моделями, работать с QuerySet, использовать Django Admin, создавать представления (Function-Based и Class-Based), URL-маршрутизацию и HTML-шаблоны, работать с формами и валидацией, реализовать CRUD, использовать CSRF-защиту и встроенную аутентификацию, написать автоматические тесты, а также сравнить архитектуру Django с FastAPI и Flask.

2. Краткие теоретические сведения

Django — высокоуровневый Python-фреймворк, который изначально предоставляет полноценный набор средств для создания веб-приложений: ORM, маршрутизацию, шаблоны, формы, аутентификацию, административную панель, middleware, сессии, защиту от CSRF, миграции и тестирование. В отличие от FastAPI (API-first) и Flask (минималистичный), Django называют batteries-included фреймворком. В проекте различают Project (конфигурация всего проекта: settings.py, urls.py) и App (отдельный функциональный компонент приложения).

Модель Django описывается классом, наследующим models.Model: поля CharField, DecimalField, IntegerField, DateField, ForeignKey и др. После изменения модели выполняются makemigrations (создание файлов миграций) и migrate (применение к базе). ORM предоставляет QuerySet: all(), filter(), exclude(), get(), order_by(), count(), exists(), агрегацию через aggregate() и annotate(), а также select_related()/prefetch_related() для оптимизации запросов к связанным объектам (избегание проблемы N+1).

URL-маршрутизация связывает адреса с представлениями: path("rooms/", views.room_list, name="room_list"). Представления бывают Function-Based (функция с параметром request) и Class-Based (ListView, DetailView, CreateView, UpdateView, DeleteView — готовые generic-классы). Шаблоны используют Django Template Language: переменные {{ }}, условия {% if %}, циклы {% for %}, наследование через {% extends %}/{% block %}. Формы (forms.Form и ModelForm) выполняют получение, преобразование и валидацию данных; CSRF-защита встроена, в HTML-форме достаточно {% csrf_token %}. Django Admin — встроенная панель управления, модели регистрируются через @admin.register с настройкой list_display, search_fields, list_filter. Аутентификация встроена: User, authentication, permissions, login/logout.

3. Индивидуальное задание

Вариант 5 — предметная область «Отель». Модели: Room (номер), RoomType (тип номера), Guest (гость), Booking (бронирование). Требуется реализовать Django-приложение: не менее 3 моделей со связями (ForeignKey: RoomType 1:N Room; Room 1:N Booking; Guest 1:N Booking), миграции, CRUD для номеров, страницы списка/детали/форм, поиск, фильтрацию (по типу и статусу), сортировку (по цене/вместимости в двух направлениях), пагинацию, статистику (aggregate/annotate), Django Admin, аутентификацию с разграничением доступа (администратор — CRUD, обычный пользователь — просмотр), формы с валидацией (цена, вместимость, пересечение дат бронирования) и не менее 15 автоматических тестов.

4. Проектирование базы данных

База данных отеля состоит из четырёх таблиц: roomtypes, rooms, guests, bookings. Схема приведена на рисунке 16.1.

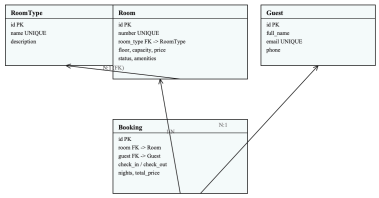


Рисунок 16.1 – ER-диаграмма базы данных

Таблица	Поля	Связи
roomtypes	id PK, name UNIQUE, description	—
rooms	id PK, number UNIQUE, room_type FK, floor, capacity, price, status, amenities	N:1 roomtypes
guests	id PK, full_name, email UNIQUE, phone	—
bookings	id PK, room FK, guest FK, check_in, check_out, nights, total_price	N:1 rooms, N:1 guests

Связи: RoomType 1:N Room (тип номера), Room 1:N Booking и Guest 1:N Booking — бронирование связывает номер и гостя. При создании бронирования в переопределённом save() автоматически вычисляются количество ночей (nights) и стоимость (total_price = nights × цена номера). Уникальные поля: number у номера и email у гостя.

5. Реализация моделей

Файл: main/models.py

```
from datetime import date

from django.db import models


def _as_date(value):
    if isinstance(value, str):
        return date.fromisoformat(value)
    return value


class RoomType(models.Model):
    """Тип номера (связь 1:N с номерами)."""

    name = models.CharField("Название", max_length=100, unique=True)
```

```
    description = models.TextField("Описание", blank=True)

    class Meta:
        verbose_name = "Тип номера"
        verbose_name_plural = "Типы номеров"

    def __str__(self):
        return self.name


class Room(models.Model):
    """Гостиничный номер — основная сущность."""

    STATUS_FREE = "free"
    STATUS_BOOKED = "booked"
    STATUS_MAINTENANCE = "maintenance"
    STATUS_CHOICES = [
        (STATUS_FREE, "Свободен"),
        (STATUS_BOOKED, "Занят"),
        (STATUS_MAINTENANCE, "На обслуживании"),
    ]

    number = models.CharField("Номер", max_length=10, unique=True)
    room_type = models.ForeignKey(
        RoomType, on_delete=models.CASCADE, related_name="rooms",
verbose_name="Тип"
    )
    floor = models.PositiveIntegerField("Этаж")
    capacity = models.PositiveIntegerField("Вместимость")
    price = models.DecimalField("Цена за ночь", max_digits=10, decimal_places=2)
    status = models.CharField("Состояние", max_length=20,
choices=STATUS_CHOICES, default=STATUS_FREE)
    amenities = models.TextField("Удобства", blank=True)

    class Meta:
        verbose_name = "Номер"
        verbose_name_plural = "Номера"
        ordering = ["number"]

    def __str__(self):
        return self.number


class Guest(models.Model):
    """Гость (связь 1:N с бронированиями)."""

    full_name = models.CharField("ФИО", max_length=200)
    email = models.EmailField("Email", unique=True)
    phone = models.CharField("Телефон", max_length=30, blank=True)

    class Meta:
        verbose_name = "Гость"
        verbose_name_plural = "Гости"
        ordering = ["full_name"]

    def __str__(self):
        return self.full_name


class Booking(models.Model):
    """Бронирование номера гостем."""

    room = models.ForeignKey(Room, on_delete=models.CASCADE,
related_name="bookings", verbose_name="Номер")
    guest = models.ForeignKey(Guest, on_delete=models.CASCADE,
related_name="bookings", verbose_name="Гость")
    check_in = models.DateField("Заезд")
    check_out = models.DateField("Выезд")
    nights = models.PositiveIntegerField("Ночей", default=1)
    total_price = models.DecimalField("Стоимость", max_digits=10,
decimal_places=2, default=0)
```

```
    class Meta:
        verbose_name = "Бронирование"
        verbose_name_plural = "Бронирования"
        ordering = ["-check_in"]

    def __str__(self):
        return f"{self.room.number} — {self.guest.full_name} ({self.check_in}…
{self.check_out})"

    def save(self, *args, **kwargs):
        check_in = _as_date(self.check_in)
        check_out = _as_date(self.check_out)
        self.nights = max((check_out - check_in).days, 1)
        self.total_price = self.nights * self.room.price
        super().save(*args, **kwargs)
```

6. Миграции

После описания моделей выполнены команды makemigrations и migrate: Django создал миграцию main/0001_initial с четырьмя таблицами и применил её к базе SQLite. Командой seed (management/commands/seed.py) база заполнена тестовыми данными: 4 типа номеров, 16 номеров, 6 гостей, 8 бронирований и администратор admin/admin123.

7. Django ORM — обязательные запросы

В приложении продемонстрированы основные операции QuerySet: all() — список всех номеров; filter() — фильтрация по типу/статусу и поиск (Q-объекты, icontains); exclude() — исключение занятых; get() — получение объекта по первичному ключу; order_by() — сортировка по цене и вместимости; count() и exists() — проверки; aggregate() — средняя/минимальная/максимальная цена и сумма выручки; annotate() — количество номеров и средняя цена по типам; select_related() и prefetch_related() — загрузка связанных объектов (типа номера, бронирований гостей в детальной странице) одним запросом вместо N+1.

8. Веб-интерфейс

Приложение предоставляет HTML-интерфейс: главную страницу со счётчиками; список номеров с пагинацией, поиском (по номеру и удобствам), фильтрацией (по типу и статусу) и сортировкой (по цене и вместимости в двух направлениях); карточку номера со связанными бронированиями; формы создания и редактирования номера; форму бронирования; список гостей и бронирований; страницу статистики. Демонстрация страниц приведена на рисунке 16.2.

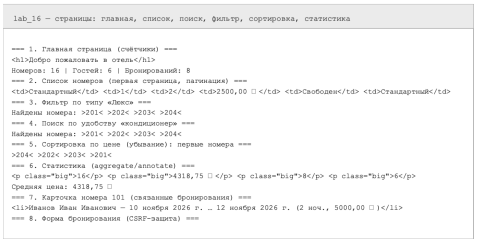


Рисунок 16.2 – Страницы приложения: главная, список, поиск, фильтр, статистика

```
main/templates/
├── base.html                    # базовый шаблон: меню, блоки, сообщения
├── main/
│   ├── index.html               # главная страница
│   ├── room_list.html           # список номеров (поиск, фильтр, сортировка,
пагинация)
│   ├── room_detail.html         # карточка номера
│   ├── room_form.html           # форма создания/редактирования
│   ├── room_confirm_delete.html
│   ├── booking_list.html / booking_detail.html / booking_form.html
│   ├── guest_list.html / guest_detail.html / guest_form.html
│   └── statistics.html          # статистика
└── registration/login.html      # форма входа
```

9. Django Admin

Все модели зарегистрированы в панели администратора: RoomAdmin с list_display (номер, тип, этаж, вместимость, цена, статус), list_filter (статус, тип, этаж), search_fields (номер, удобства) и list_editable; GuestAdmin с поиском по имени и email; BookingAdmin с list_filter по дате заезда и номеру, search_fields по гостю и номеру, autocomplete_fields. Администратор создаётся командой createsuperuser или автоматически командой seed.

10. Формы и валидация

ModelForm-ы выполняют валидацию на сервере: RoomForm проверяет положительность цены и вместимости; BookingForm проверяет, что дата выезда позже даты заезда, и что номер не пересекается с уже существующими бронированиями (запрос filter с условием пересечения интервалов). Все формы используют защиту CSRF (тег {% csrf_token %}): Django отклоняет POST-запросы без токена, что защищает от подделки межсайтовых запросов.

Файл: main/forms.py

```
from django import forms
```

```
from .models import Booking, Guest, Room


class RoomForm(forms.ModelForm):
    """ModelForm для номера: валидация цены и вместимости."""

    class Meta:
        model = Room
        fields = ["number", "room_type", "floor", "capacity", "price", "status",
"amenities"]
        widgets = {
            "amenities": forms.Textarea(attrs={"rows": 2}),
        }

    def clean_price(self):
        price = self.cleaned_data["price"]
        if price <= 0:
            raise forms.ValidationError("Цена должна быть положительной")
        return price

    def clean_capacity(self):
        capacity = self.cleaned_data["capacity"]
        if capacity <= 0:
            raise forms.ValidationError("Вместимость должна быть больше нуля")
        return capacity


class BookingForm(forms.ModelForm):
    """ModelForm для бронирования: гость, номер, даты."""

    class Meta:
        model = Booking
        fields = ["room", "guest", "check_in", "check_out"]
        widgets = {
            "check_in": forms.DateInput(attrs={"type": "date"}),
            "check_out": forms.DateInput(attrs={"type": "date"}),
        }

    def clean(self):
        cleaned = super().clean()
        check_in = cleaned.get("check_in")
        check_out = cleaned.get("check_out")
        if check_in and check_out and check_out <= check_in:
            raise forms.ValidationError("Дата выезда должна быть позже даты
заезда")
        room = cleaned.get("room")
        if room and check_in and check_out:
            overlaps = Booking.objects.filter(
                room=room, check_in__lt=check_out, check_out__gt=check_in
            )
            if self.instance.pk:
                overlaps = overlaps.exclude(pk=self.instance.pk)
            if overlaps.exists():
                raise forms.ValidationError("Номер уже забронирован на эти
даты")
        return cleaned


class GuestForm(forms.ModelForm):
    class Meta:
        model = Guest
        fields = ["full_name", "email", "phone"]
```

11. Views и права доступа

CRUD реализован сочетанием подходов: Function-Based Views (room_list, room_detail, room_create, room_update — с декоратором login_required и явной проверкой is_staff/is_superuser, при отсутствии прав возвращается 403) и Class-

Based Views (RoomDeleteView, BookingCreateView, BookingListView, BookingDetailView, GuestListView и др. — с LoginRequiredMixin и UserPassesTestMixin). Сравнение: CBV сокращают типовой код (список с пагинацией — 3 строки), но FVB нагляднее для нестандартной логики; в проекте используются оба подхода.

Файл: main/views.py (фрагмент)

```
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.core.paginator import Paginator
from django.db.models import Avg, Count, Max, Min, Q, Sum
from django.http import HttpResponseForbidden
from django.shortcuts import get_object_or_404, redirect, render
from django.urls import reverse_lazy
from django.views.generic import CreateView, DeleteView, DetailView, ListView,
UpdateView

from .forms import BookingForm, GuestForm, RoomForm
from .models import Booking, Guest, Room, RoomType


def is_admin(user):
    return user.is_authenticated and (user.is_staff or user.is_superuser)


# ---------- главная и статистика ----------
def index(request):
    rooms_total = Room.objects.count()
    rooms_free = Room.objects.filter(status=Room.STATUS_FREE).count()
    guests_total = Guest.objects.count()
    bookings_total = Booking.objects.count()
    return render(request, "main/index.html", {
        "rooms_total": rooms_total,
        "rooms_free": rooms_free,
        "guests_total": guests_total,
        "bookings_total": bookings_total,
    })


def statistics(request):
    stats = {
        "rooms_total": Room.objects.count(),
        "rooms_free": Room.objects.filter(status=Room.STATUS_FREE).count(),
        "rooms_booked": Room.objects.filter(status=Room.STATUS_BOOKED).count(),
        "avg_price": Room.objects.aggregate(avg=Avg("price"))["avg"] or 0,
        "max_price": Room.objects.aggregate(max=Max("price"))["max"] or 0,
        "min_price": Room.objects.aggregate(min=Min("price"))["min"] or 0,
        "bookings_total": Booking.objects.count(),
        "revenue": Booking.objects.aggregate(total=Sum("total_price"))["total"]
or 0,
        "guests_total": Guest.objects.count(),
    }
    by_type = (
        RoomType.objects.annotate(count=Count("rooms"),
avg_price=Avg("rooms__price"))
        .order_by("-count")
    )
    return render(request, "main/statistics.html", {"stats": stats, "by_type":
by_type})


# ---------- CRUD номеров (FVB) ----------
def room_list(request):
```

```
query = request.GET.get("q", "")
room_type = request.GET.get("type", "")
status = request.GET.get("status", "")
sort = request.GET.get("sort", "number")

rooms = Room.objects.select_related("room_type")
if query:
```

12. Аутентификация

Использована встроенная система пользователей Django: страница входа на /accounts/login/, выход через POST-форму, текущий пользователь выводится в меню (user.username, user.is_staff). Разграничение доступа: любой посетитель видит главную, список, карточки и статистику; создание, изменение и удаление номеров и бронирований доступно только администратору (is_staff/is_superuser) — при попытке обычного пользователя выполнить операцию возвращается 403 Forbidden. Пароли хранятся в зашифрованном виде (PBKDF2-хеширование), что исключает их хранение в открытом виде.

13. Тестирование

Написано 25 автоматических тестов (django.test.TestCase): главная страница, список, детальная страница, 404, статистика, поиск, фильтрация по типу, сортировка, пагинация, список гостей и бронирований, CSRF-защита, обязательная аутентификация для создания, создание номера администратором, валидация неположительной цены, запрет для обычного пользователя (403), редактирование и удаление, создание бронирования с расчётом ночей и стоимости, отказ при пересечении дат, а также тесты ORM (filter/exclude, get/exists, order_by, aggregate/annotate, связи, select_related). Результат выполнения приведён на рисунке 16.3.

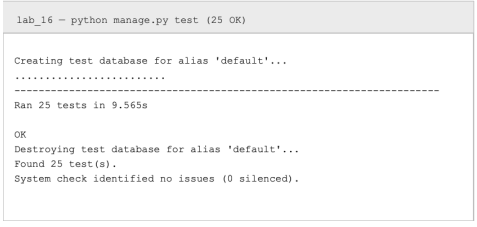


Рисунок 16.3 – Результаты выполнения тестов (25 OK)

14. Сравнение Django, Flask и FastAPI

Возможность	Django	Flask	FastAPI
ORM	встроенная	нет	нет
Templates	встроены	поддерживаются	не основной сценарий
Admin	встроен	нет	нет
Forms	встроены	расширения	обычно Pydantic
REST API	через DRF	удобно	основной сценарий
OpenAPI	через DRF	расширения	встроено
Async	поддерживается	поддерживается	сильная сторона
Authentication	встроенная	расширения	отдельно
Batteries included	высокий	минимальный	средний

Django предпочтителен для полнофункциональных веб-приложений с админкой, формами и аутентификацией «из коробки»; FastAPI — для API-first проектов с высокой производительностью и автоматической OpenAPI-документацией; Flask удобен для небольших приложений, где нужен минимализм и свобода выбора компонентов.

15. Вывод

В ходе лабораторной работы разработано полноценное Django-приложение «Отель»: спроектирована база данных из четырёх моделей со связями (RoomType 1:N Room, Room 1:N Booking, Guest 1:N Booking), созданы миграции, заполнены тестовые данные, реализован CRUD с сочетанием Function-Based и Class-Based Views, поиск, фильтрация, сортировка и пагинация средствами Django ORM, статистика через aggregate() и annotate(), формы с валидацией и CSRF-защитой, административная панель, аутентификация с разграничением прав и 25 автоматических тестов.

Выполненная работа позволила сравнить изученные ранее фреймворки: Django даёт готовую инфраструктуру (ORM, Admin, Forms, Templates, Auth) и подходит для полноценных веб-приложений, тогда как Flask минималистичен, а FastAPI ориентирован на API. Полученные навыки — моделирование, QuerySet, миграции, шаблоны, формы, аутентификация — являются основой для дальнейшей разработки на Django.